

```

bhavitha@SAIBHAVITHA: ~
× + v
bhavitha@SAIBHAVITHA:~$ nano project.c
bhavitha@SAIBHAVITHA:~$ gcc project.c -o shell
bhavitha@SAIBHAVITHA:~$ ./shell

***** ARGUMENT PASSING IN LINUX PROCESS EXECUTION *****
1. fork + execve custom argv/envp
2. Mini shell (quotes + multiple arguments)
3. /proc cmdline/envIRON inspection
4. Anonymous pipe IPC
5. getopt() option parsing
6. OS layered execution trace (C01)
0. Exit
Choose: |
```

```

0. Exit
Choose: 1

=====
C02: fork() + execve() + waitpid() + exit()
=====
Parent constructs custom argv[] and envp[], then forks.
[PARENT] fork() created child PID=483
[CHILD before execve] PID=483, calling execve()...

=====
EXECUTED CHILD: argc / argv / envp
=====
PID: 483
Parent PID: 479
argc = 3
argv[0] = "hello"
argv[1] = "Linux"
argv[2] = "argument passing"

Selected environment variables:
OS_PROJECT=ArgumentPassingDemo
TEAM=3
ARG_SOURCE=parent-created-envp
[PARENT] waitpid(): child exited with status 0
```

Choose: 2

=====

C01 + C02: Mini Shell (user-space shell -> fork -> execvp -> waitpid)

=====

Type commands such as:

```
echo hello world
echo "Operating Systems" project
ls -l
pwd
```

Type 'exit' to return to the main menu.

os-shell\$ ls

[shell parent] waiting for PID=488

[shell child] execvp("ls")

1.c	15.c	20.c	3.c	8.c	client_fifo	example.txt	hmm.c	paging.c
10.c	16.c	21.c	4.c	88.c	coltoco3	hardlink.txt	li.c	prog.c
11.c	17.c	22.c	5.c	9.c	coltoco3.c	harsha.c	multi.c	prog09.c
12.c	18.c	23.c	6.c	99.c	coltoco3.save	harsha.c.save	original.txt	prog10.c
13.c	19.c	24.c	7.c	Addressspacs.c	combined.txt	hello	os.c	prog100
14.c	2.c	25.c	8	child.c	destination.txt	hello.c	output.txt	prog100.c

[shell parent] exit status=0

os-shell\$ |

Choose: 3

=====

C01/Abstract: /proc inspection of arguments and environment

=====

[PARENT] Child PID=489 is running. Inspecting /proc...

=====

CHILD WAITING FOR /proc INSPECTION

=====

PID: 489

Received 3 argument(s)

argv[0] = "one"

argv[1] = "two words"

argv[2] = "three"

Sleeping for 5 seconds so the parent can inspect /proc.

/proc/489/cmdline:

/home/bhavitha/shell --inspect-child one two words three

/proc/489/envron:

OS_PROJECT=ArgumentPassingDemo TEAM=3 ARG_SOURCE=execve-environment

[PARENT] /proc demonstration complete.

0. Exit

Choose: 5

=====

getopt() structured option parsing

=====

Parsed name = Saranya

Parsed count = 3

[1] Hello Saranya

[2] Hello Saranya

[3] Hello Saranya

Choose: 6

=====

C01: Layered execution trace

=====

Kernel-mode proof-of-concept: getpid() via libc=479, via syscall=479

1. User types a command in the terminal.
2. Shell (user mode) reads and tokenizes the command.
3. Shell calls fork() through the C library/syscall interface.
4. Child calls execvp()/execve(); kernel replaces the child process image.
5. Kernel prepares argc/argv/envp for the new program.
6. New program starts at its C runtime and reaches main(argc, argv, envp).
7. Parent calls waitpid() and receives the child's termination status.

Layer view:

User -> Shell -> C library/syscall interface -> Kernel -> Process image -> C runtime -> main()

01-2024

Choose: 0

Exiting project.

bhavitha@SAIBHAVITHA:~\$ |